



Documento de Análisis del Sistema

Versión 1.0

Fecha 2025-09-19

Preparado para:

[Universidad de Sevilla \(Escuela Técnica Superior de Ingeniería Informática\):](#)

Preparado por:

[Grupo de desarrollo del TFG](#)

Índice

1 [Introducción](#)

- 1.1 [Alcance del proyecto](#)
- 1.2 [Participantes en el proyecto](#)
 - 1.2.1 [Organizaciones participantes](#)
 - 1.2.2 [Personas participantes](#)
- 1.3 [Objetivos del proyecto](#)

2 [Arquitectura lógica del sistema](#)

- 2.1 [Diagramas de la arquitectura del sistema](#)
- 2.2 [Descripción de la arquitectura del sistema](#)

3 [Modelo de clases del sistema](#)

- 3.1 [Diagramas de clases del sistema](#)
- 3.2 [Descripción de las clases del sistema](#)
- 3.3 [Diagramas de estado de las clases del sistema](#)

4 [Modelo de casos de uso del sistema](#)

- 4.1 [Diagramas de secuencia del sistema](#)
- 4.2 [Descripción de los diagramas de secuencia del sistema](#)

5 [Interfaz de usuario del sistema](#)

- 5.1 [Diagramas de navegación del sistema](#)
- 5.2 [Esquemas de la interfaz de usuario del sistema](#)
- 5.3 [Descripción de la interfaz de usuario del sistema](#)

6 [Interfaz de servicios del sistema](#)

- 6.1 [Diagramas de la interfaz de servicios del sistema](#)
- 6.2 [Descripción de la interfaz de servicios del sistema](#)
- 6.3 [Servicios consumidos por el sistema](#)

7 [Información sobre trazabilidad](#)

A [Glosario de acrónimos y abreviaturas](#)

1 Introducción

1.1 Alcance del proyecto

El objetivo principal del proyecto es diseñar e implementar un sistema software que facilite la gestión integral de entregables en asignaturas universitarias. El sistema debe permitir a los profesores crear actividades dirigidas a grupos específicos de estudiantes y proporcionar a los alumnos un mecanismo sencillo y centralizado para realizar sus entregas.

Para alcanzar este propósito general, se han definido los siguientes objetivos específicos:

- **O1. Centralización de la estructura académica:** Permitir la creación y gestión de asignaturas que contengan múltiples grupos de estudiantes, reflejando la realidad organizativa de los cursos universitarios.
- **O2. Flexibilidad en la definición de actividades:** Habilitar a los docentes para definir actividades con descripciones detalladas, instrucciones, ficheros base adjuntos y fechas de entrega estrictas.
- **O3. Optimización del proceso de entrega:** Implementar un formulario de entrega simplificado para los estudiantes que incluya verificación de ficheros y, opcionalmente, el uso de enlaces únicos para facilitar el acceso.
- **O4. Automatización de la organización documental:** Desarrollar un sistema que organice automáticamente las entregas recibidas en una jerarquía de carpetas coherente (Asignatura > Grupo > Actividad > Estudiante), eliminando la carga manual de clasificación por parte del profesor.
- **O5. Interoperabilidad y Escalabilidad:** Diseñar el sistema con una arquitectura modular que permita la integración futura con servicios de almacenamiento en la nube (como OneDrive o Nextcloud) y su comunicación mediante APIs REST.

1.2 Participantes en el proyecto

1.2.1 Organizaciones participantes

 Organización	Universidad de Sevilla (Escuela Técnica Superior de Ingeniería Informática):
Dirección	E.T.S. Ingeniería Informática Avda. Reina Mercedes s/n 41012 Sevilla
Sitio web	https://www.us.es

 Organización	Grupo de desarrollo del TFG
Dirección	E.T.S. Ingeniería Informática Avda. Reina Mercedes s/n 41012 Sevilla
Teléfono	647658345
Correo-e	mancalrod@alum.us.es

1.2.2 Personas participantes

 Participante	Calderón Rodríguez, Manuel María
Organización	• Grupo de desarrollo del TFG
Rol	Desarrollador Full Stack
Categoría	Desarrollador
Correo-e	mancalrod@alum.us.es

 Participante	Márquez Gutiérrez, José Manuel
Organización	• Grupo de desarrollo del TFG
Rol	Desarrollador Full Stack
Categoría	Desarrollador
Correo-e	josmargut@alum.us.es

 Participante	Gutiérrez Fernandez, Antonio Manuel
Organización	Universidad de Sevilla (Escuela Técnica Superior de Ingeniería Informática):
Rol	Tutor TFG (cliente)
Categoría	Cliente
Correo-e	amgutierrez@us.es

1.3 Objetivos del proyecto

El objetivo principal del proyecto es diseñar e implementar un sistema software que facilite la gestión integral de entregables en asignaturas universitarias. El sistema debe permitir a los profesores crear actividades dirigidas a grupos específicos de estudiantes y proporcionar a los alumnos un mecanismo sencillo y centralizado para realizar sus entregas.

Para alcanzar este propósito general, se han definido los siguientes objetivos específicos:

- **O1. Centralización de la estructura académica:** Permitir la creación y gestión de asignaturas que contengan múltiples grupos de estudiantes, reflejando la realidad organizativa de los cursos universitarios.
- **O2. Flexibilidad en la definición de actividades:** Habilitar a los docentes para definir actividades con descripciones detalladas, instrucciones, ficheros base adjuntos y fechas de entrega estrictas.
- **O3. Optimización del proceso de entrega:** Implementar un formulario de entrega simplificado para los estudiantes que incluya verificación de ficheros y, opcionalmente, el uso de enlaces únicos para facilitar el acceso.
- **O4. Automatización de la organización documental:** Desarrollar un sistema que organice automáticamente las entregas recibidas en una jerarquía de carpetas coherente (Asignatura > Grupo > Actividad > Estudiante), eliminando la carga manual de clasificación por parte del profesor.
- **O5. Interoperabilidad y Escalabilidad:** Diseñar el sistema con una arquitectura modular que permita la integración futura con servicios de almacenamiento en la nube (como OneDrive o Nextcloud) y su comunicación mediante APIs REST.

2 Arquitectura lógica del sistema

2.1 Diagramas de la arquitectura del sistema

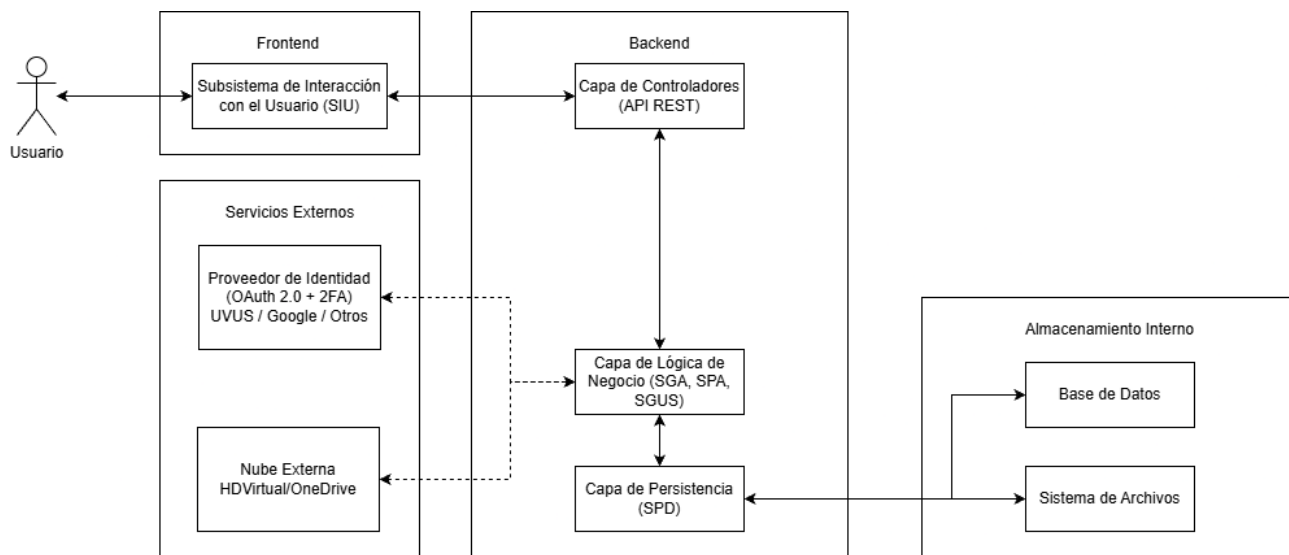


Figura 1: Arquitectura del sistema

2.2 Descripción de la arquitectura del sistema

Nuestro sistema software sigue una arquitectura basada en el patrón **Modelo Vista Controlador (MVC)**, diseñado bajo un modelo desacoplado que distingue claramente entre el cliente (Frontend) y el servidor (Backend).

Esta separación permite una mayor flexibilidad tecnológica y facilita la integración con servicios externos de almacenamiento y una gestión de identidad segura mediante protocolos estándar como **OAuth 2.0**.

La estructura lógica del sistema se divide en los siguientes bloques principales:

Frontend (Capa de Presentación):

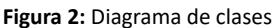
Backend (Lógica y Control):

- **Capa de Controladores:** Gestiona la entrada de peticiones desde el Frontend y delega las tareas a los subsistemas lógicos.
- **Capa de Lógica de Negocio:** Implementa las reglas del sistema mediante tres componentes clave:
 - **SGUS (Seguridad):** Controla roles y permisos .
 - **SGA (Académica):** Gestiona asignaturas, actividades y evaluaciones .
 - **SPA (Archivos):** Ejecuta la validación, renombrado y organización automática de los ficheros entregados .
- **Capa de Persistencia (SPD):** Abstrae el almacenamiento de datos, interactuando tanto con la **Base de Datos Relacional** (para información estructurada) como con el **Sistema de Archivos** físico (para los documentos) .

El sistema se integra con proveedores externos para delegar funcionalidades críticas:

- **Gestión de Identidad y Acceso (OAuth 2.0 + 2FA):** Para la autenticación, el sistema utiliza el protocolo **OAuth 2.0** . Esto permite integrar proveedores de identidad como **UVUS** o plataformas genéricas (Google), reforzando la seguridad mediante **Doble Factor de Autenticación (2FA)**.
- **Almacenamiento en Nube:** Se habilita la interoperabilidad con servicios externos (como Nextcloud o OneDrive) para la sincronización y respaldo de la jerarquía de carpetas generada por el sistema .

3.1 Diagramas de clases del sistema



3.2 Descripción de las clases del sistema

 ENT-001	Usuario
<pre>/** * Esta clase entidad representa los usuarios de la aplicación. */ class Usuario { nombre: string teléfono: string correo electrónico: string contraseña: string esAdmin: boolean // Verifica si el usuario es admin. }</pre>	
 ENT-002	Profesor
<pre>/** * Esta clase entidad representa que usuario es profesor de que curso. */ class Profesor { }</pre>	
 ENT-003	Estudiante
<pre>/** * Esta clase entidad representa que usuario es estudiante de que grupo. */ class Estudiante { }</pre>	
 ENT-004	Curso
<pre>/** * Esta clase entidad representa el curso en el que se encuentran los profesores. */ class Curso { título: string descripción: string // Una descripción del curso. }</pre>	
 ENT-005	Grupo
<pre>/** * Esta clase entidad representa el grupo de el curso en el que se encuentran los profesores. */ class Grupo { título: string }</pre>	
 ENT-006	Actividad
<pre>/** * Esta clase entidad representa la actividad que solo podrán crear los profesores del curso. */ class Actividad { título: string descripción: string tipoActividad: tipoActividad fechaDeCreacion: date fechaLimite: date fechaInicio: date visibilidad: visibilidad }</pre>	

ENT-007	Material
<pre> /** Esta clase entidad representa tanto los materiales de ayuda que puede aportar el profesor tanto en actividades como entregables y estudiantes cuando realiza una entrega. */ class Material { tipoMaterial: tipoMaterial ruta: string } </pre>	

ENT-008	Entregable
<pre> /** Esta clase entidad representa los subapartados de las actividades. */ class Entregable { titulo: string descripcion: string fechaLimite: date fechaInicio: date notaMáxima: double calificacion: double tipoDeArchivoEsperado: tipoMaterial } </pre>	

ENT-009	Entrega
<pre> /** Esta clase entidad representa la entrega que realiza un alumno a un entregable. */ class Entrega { nombre: string version: string } </pre>	

ENT-010	Feedback
<pre> /** Esta clase entidad representa los mensajes que realiza un profesor a un entregable que realizó un profesor. */ class Feedback { comentario: string } </pre>	

3.3 Diagramas de estado de las clases del sistema

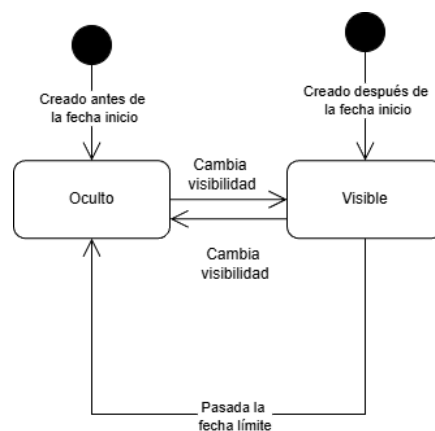


Figura 3: Diagrama de estado de las actividades

4 Modelo de casos de uso del sistema

4.1 Diagramas de secuencia del sistema

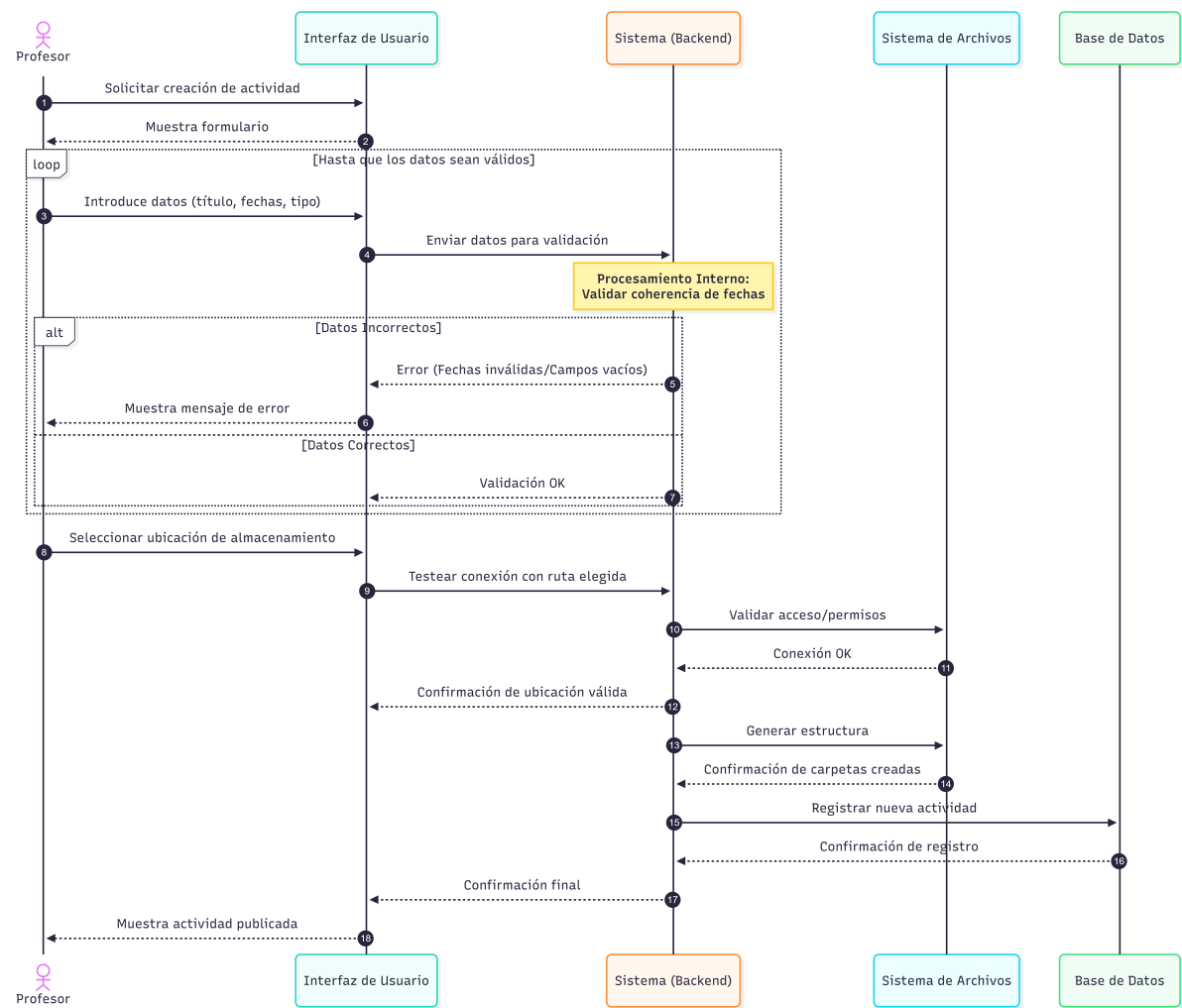


Figura 4: Crear Actividad

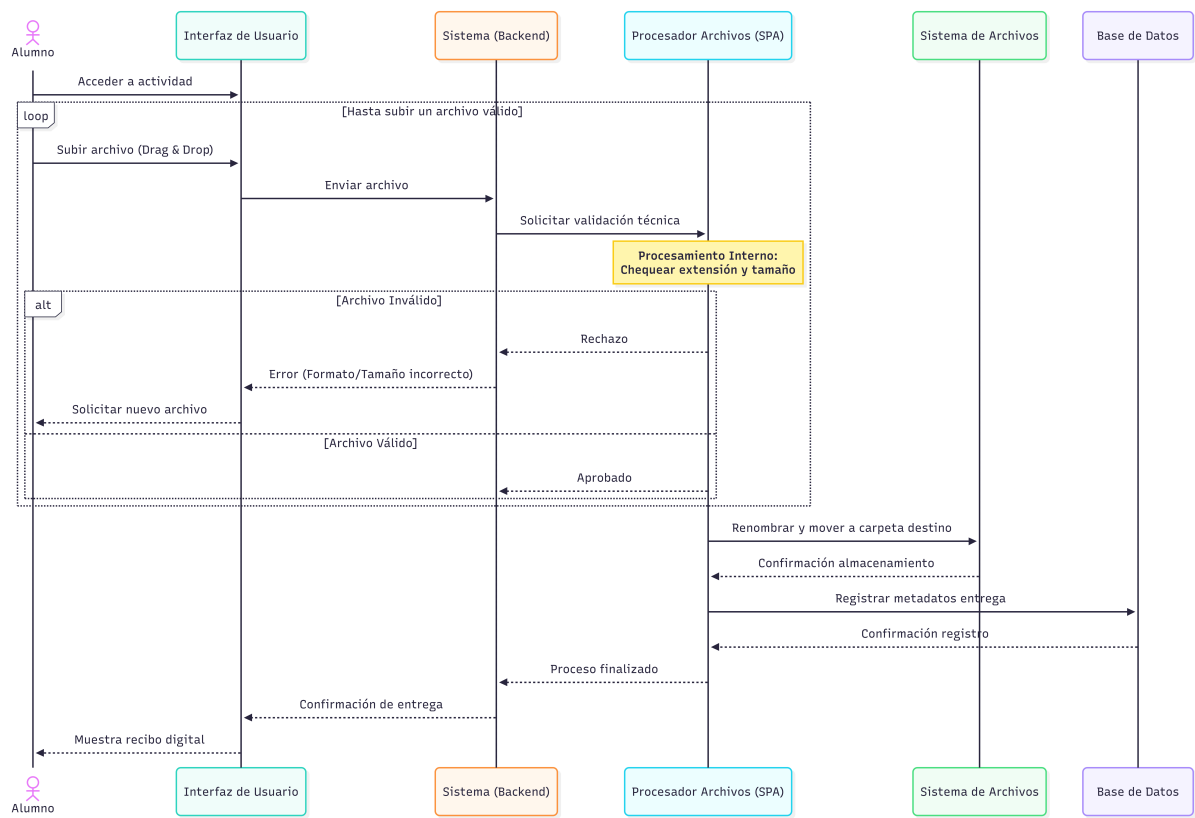


Figura 5: Realizar Entrega

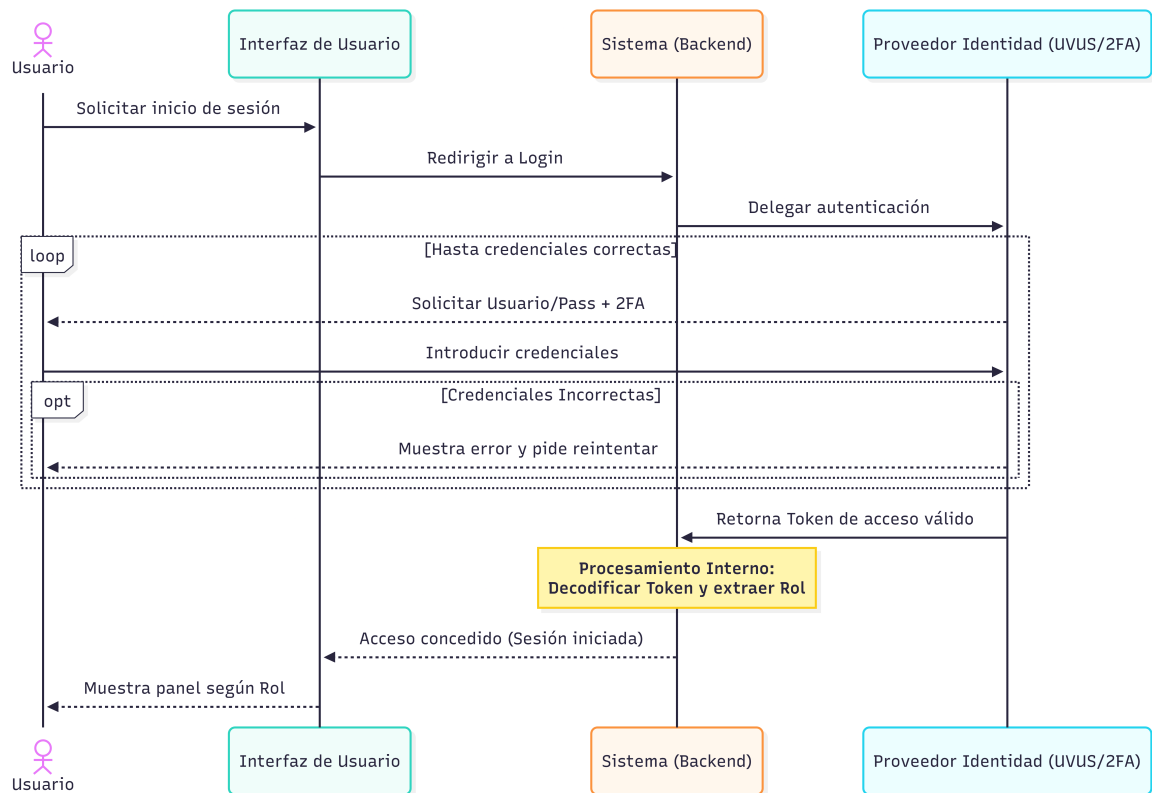


Figura 6: Autenticación y Acceso

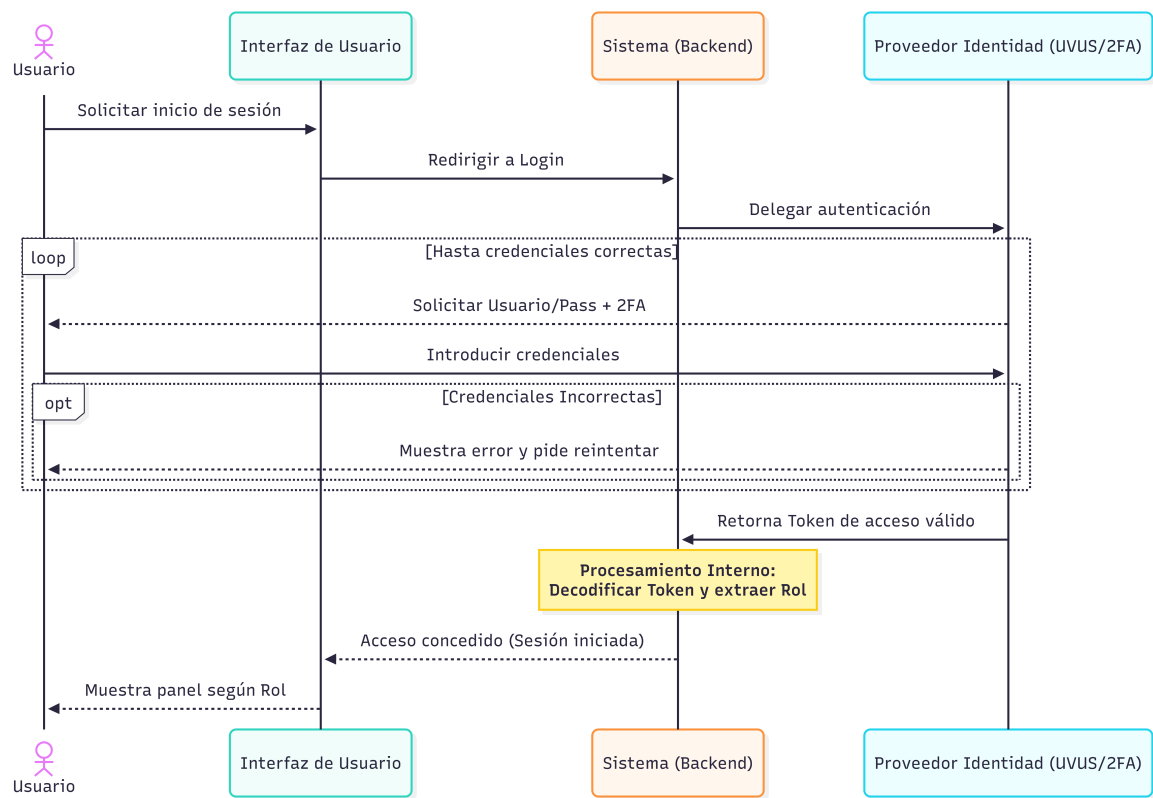


Figura 7: Evaluación y Feedback

4.2 Descripción de los diagramas de secuencia del sistema

A continuación se describen los flujos de interacción principales representados en los diagramas anteriores, correspondientes a los procesos críticos del negocio:

- **Creación de Actividad:** Este diagrama modela la interacción mediante la cual el profesor configura una nueva tarea académica. El proceso incluye un bucle de validación en el Backend que impide avanzar si las fechas son incoherentes o faltan datos obligatorios. Una vez validados los datos, el sistema orquesta la creación física de la estructura de directorios en el servidor antes de confirmar el registro en la base de datos .
- **Realización de Entrega:** Representa el proceso crítico de subida de archivos por parte del alumno. El diagrama destaca el papel del **Subsistema de Procesamiento de Archivos (SPA)**, que actúa dentro de un bucle de interacción validando en tiempo real los requisitos técnicos (extensión, tamaño, virus). Solo cuando el archivo supera estas validaciones, el sistema procede a su renombrado automático y almacenamiento definitivo .
- **Autenticación y Acceso:** Muestra el flujo de seguridad externalizado. El usuario interactúa repetidamente con el proveedor de identidad (UVUS/OAuth) hasta verificar correctamente sus credenciales y superar el **Doble Factor de Autenticación (2FA)**. Tras el éxito, el sistema recibe un token seguro que le permite identificar el rol del usuario y concederle acceso al panel correspondiente .
- **Evaluación y Feedback:** Describe el flujo de trabajo del profesor para calificar. El sistema recupera el archivo previamente organizado y, tras la introducción de la nota y los comentarios por parte del docente, se encarga de persistir la información y generar una notificación asíncrona para informar al estudiante de su nueva calificación .

5 Interfaz de usuario del sistema

5.1 Diagramas de navegación del sistema

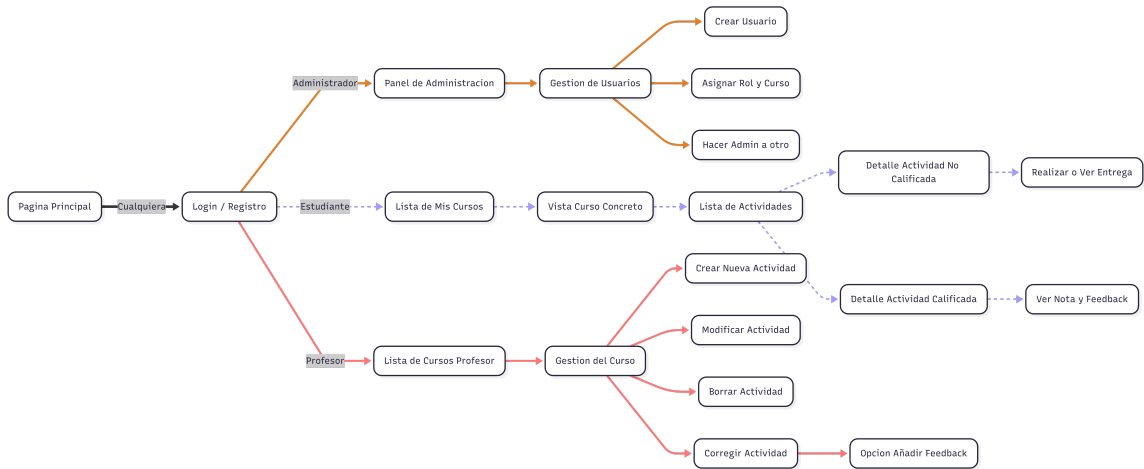


Figura 8: Diagrama de navegabilidad

5.2 Esquemas de la interfaz de usuario del sistema

Iniciar sesión

Nombre de usuario:

john doe

Contraseña:

Iniciar sesión

[¿Olvido su contraseña?](#)

Nuevo usuario

Registrarse

Figura 9: Mock up 1

Registrarse

Correo electrónico:

john doe@gmail.com

Teléfono:

123456789

Nombre de usuario:

john doe

Contraseña:

Finalizar

Figura 10: Mock up 2

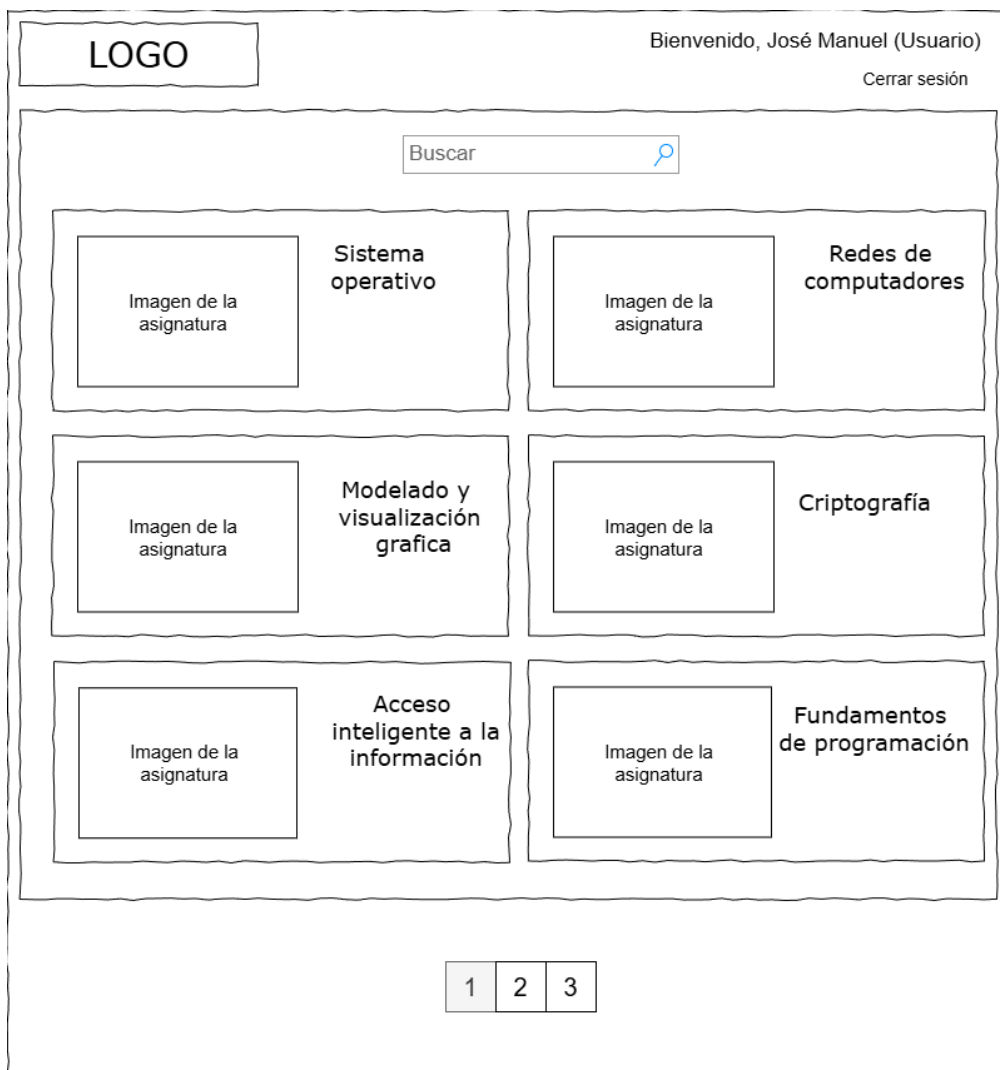


Figura 11: Mock up 3

LOGO	Bienvenido, José Manuel (Usuario) Cerrar sesión	
Imagen de la asignatura		Sistema operativo
Buscar		Enseñar entregables no disponibles ☐
Prueba de comandos Fecha de vencimiento: 7/2/2027 20:11		Iniciar entrega No entregado
1		
Entregables disponibles		Calificaciones

Figura 12: Mock up 4

LOGO

Bienvenido, José Manuel (Usuario)
Cerrar sesión

Imagen de la
asignatura

Sistema operativo

Buscar

Enseñar entregables no disponibles

Prueba de comandos

Examen para ver que conocimientos se conocen de Linux

Fecha de vencimiento: 7/2/2027 20:11

Primer intento: 7/2/2026 20:11

Iniciar entrega

Prueba de Docker

Examen para ver que conocimientos se conocen de Docker

Fecha de vencimiento: 7/2/2025 20:11

No entregado a tiempo

Prueba de Docker (Opcional)

Examen para ver que conocimientos se conocen de Docker

Fecha de vencimiento: 7/2/2024 20:11

No entregado a tiempo

1

2

3

Entregables disponibles

Calificaciones

Figura 13: Mock up 5

LOGO

Bienvenido, José Manuel (Usuario)
Cerrar sesión

Imagen de la
asignatura

Sistema operativo

Buscar



Prueba de comandos

Examen para ver que conocimientos se conocen de Linux

Fecha de vencimiento: 7/2/2024 20:11

Calificación: 4/5

Ver Feedback



Prueba de Docker

Examen para ver que conocimientos se conocen de Docker

Fecha de vencimiento: 7/2/2024 20:11

Calificación: 0/10

No entregado

1

Entregables disponibles

Calificaciones

Figura 14: Mock up 6

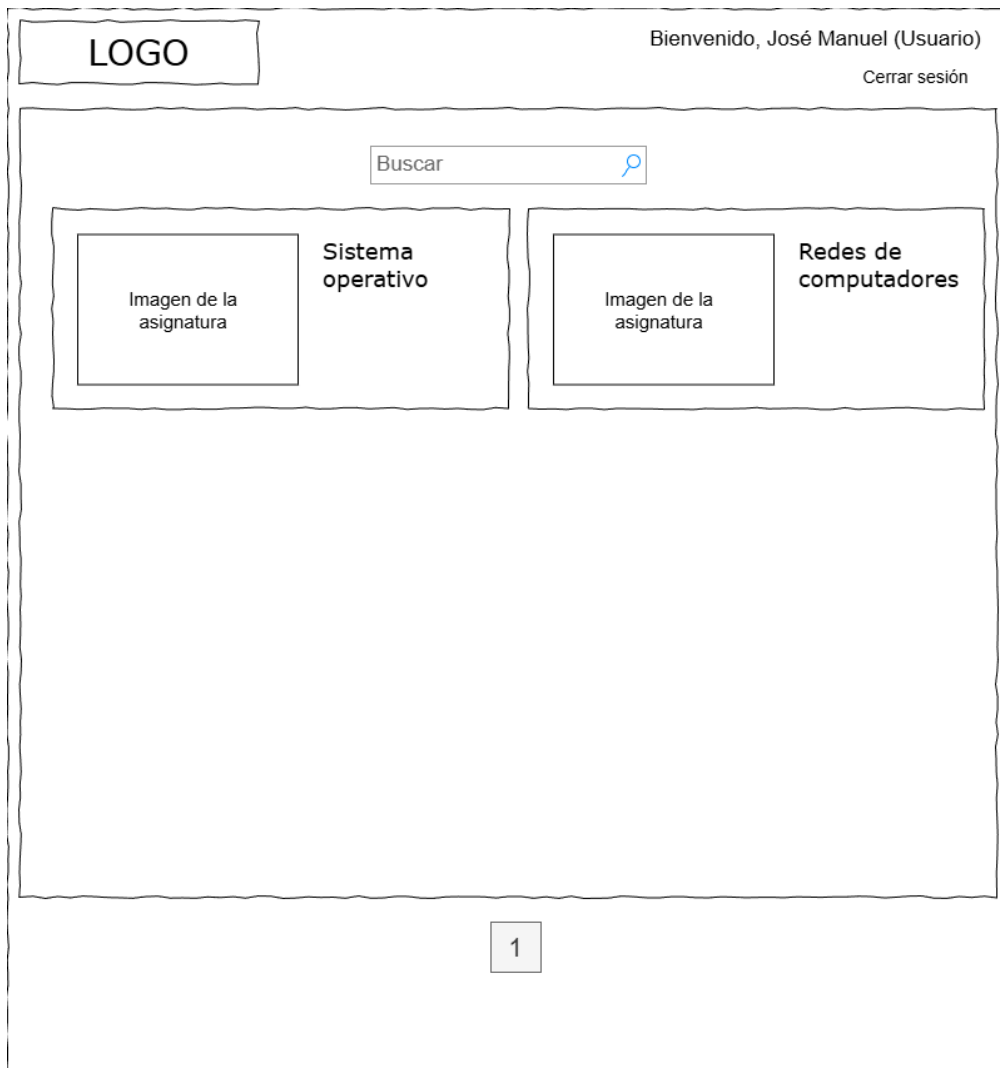


Figura 15: Mock up 7

LOGO	Bienvenido, José Manuel (Usuario) Cerrar sesión
Imagen de la asignatura	
Sistema operativo	
Seleccionar grupo ▼	Buscar 🔍
Crear entregable	
 Prueba de comandos Examen para ver que conocimientos se conocen de Linux Fecha de vencimiento: 7/2/2027 20:11 Editar actividad Revisar entregados Alumnos por entregar: 12 / 45	
 Prueba de Docker Examen para ver que conocimientos se conocen de Docker Fecha de vencimiento: 7/2/2027 20:11 Editar actividad Revisar entregados Alumnos por entregar: 42 / 42	
 Prueba de máquinas virtuales Examen para ver que conocimientos se conocen sobre máquinas virtuales Fecha de vencimiento: 7/2/2027 20:11 Editar actividad Revisar entregados Alumnos por entregar: 0 / 45	
123	
Asignaturas	Estudiantes

Figura 16: Mock up 8

LOGO

Bienvenido, José Manuel (Usuario)
Cerrar sesión

Sistema operativo

 Oculto para los estudiantes ▼

Titulo

Descripción

Tipo de puntuación

Puntos ▼

Puntuación máxima

100

Fecha final

11/9/25 

20:34 

Seleccionar grupo(s) ▼

Crear actividad

Asignaturas

Estudiantes

Figura 17: Mock up 9

LOGO

Bienvenido, José Manuel (Usuario)
Cerrar sesión

Sistema operativo

 Oculto para los estudiantes ▼

Título

Descripción

Tipo de puntuación

Puntos ▼

Puntuación máxima

100

Fecha final

11/9/25 

20:34 

Modificar actividad

Eliminar actividad

Asignaturas

Estudiantes

Figura 18: Mock up 10

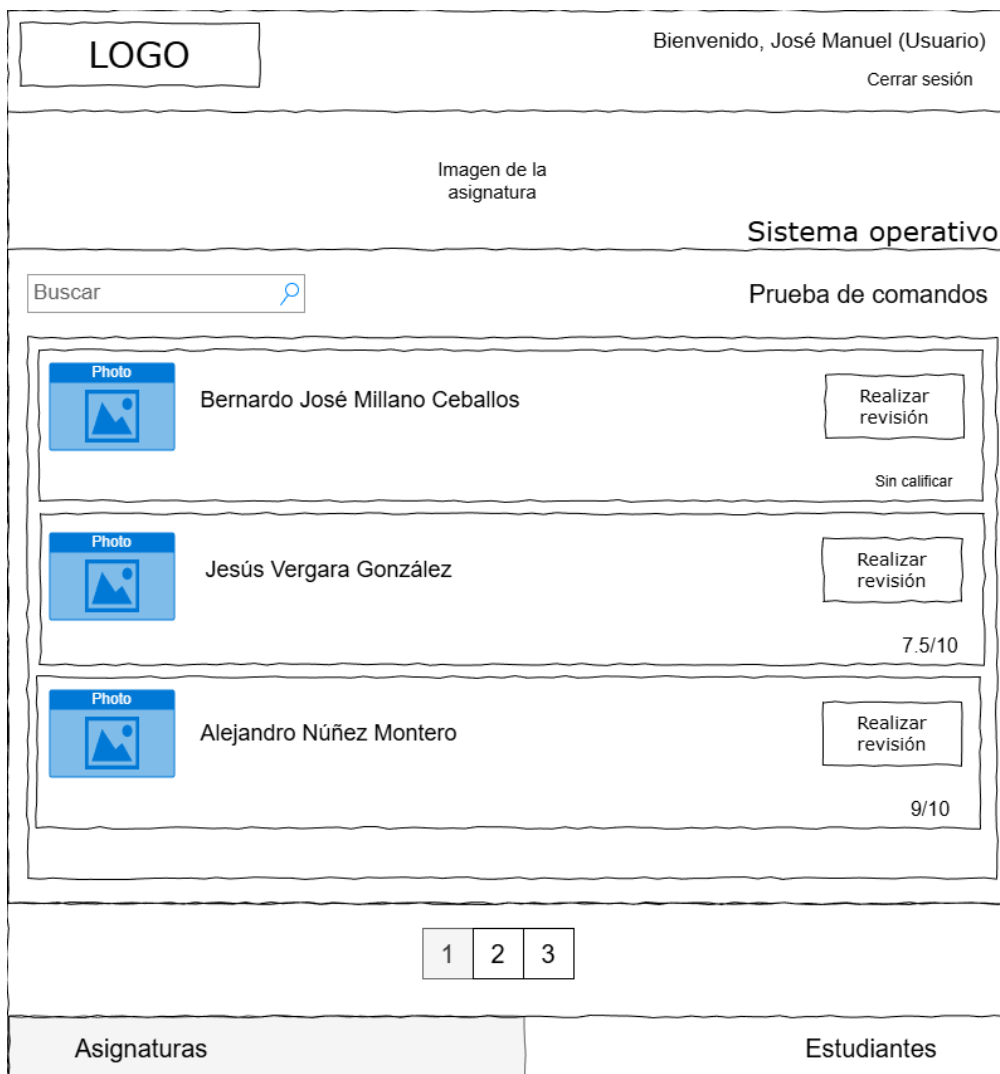


Figura 19: Mock up 11

LOGO	Bienvenido, José Manuel (Usuario) Cerrar sesión
Imagen de la asignatura	
Sistema operativo	
Prueba de comandos	Fecha de vencimiento: 7/2/2024 20:11
Detalles: Examen para ver que conocimientos se conocen de Linux	
Memoria:  Memoria_VNK5300.pdf Descargar	
Una función random (o función aleatoria) es aquella que genera resultados que no siguen un patrón fijo, sino que varían de manera impredecible dentro de un rango definido. En programación, suele referirse a funciones que devuelven números pseudoaleatorios, es decir, valores que parecen aleatorios pero en realidad son generados por un algoritmo matemático a partir de una "semilla" inicial.	
Sin calificar	Calificar Poner comentario
Asignaturas	Estudiantes

Figura 20: Mock up 12

LOGO		Bienvenido, José Manuel (Usuario) Cerrar sesión	
Imagen de la asignatura			
Sistema operativo			
Prueba de comandos		Fecha de vencimiento: 7/2/2024 20:11	
Detalles:		Primera entrega	
Examen para ver que conocimientos se conocen de Linux			
Memoria:			
Entregar			
Asignaturas		Estudiantes	

Figura 21: Mock up 13

LOGO	Bienvenido, José Manuel (Usuario) Cerrar sesión
Imagen de la asignatura	
Sistema operativo	
Prueba de comandos	Fecha de vencimiento: 7/2/2024 20:11
Primera entrega	
Detalles:	
Examen para ver que conocimientos se conocen de Linux	
Memoria:	 Memoria_VNK5300.pdf Eliminar
Una función random (o función aleatoria) es aquella que genera resultados que no siguen un patrón fijo, sino que varían de manera impredecible dentro de un rango definido. En programación, suele referirse a funciones que devuelven números pseudoaleatorios, es decir, valores que parecen aleatorios pero en realidad son generados por un algoritmo matemático a partir de una "semilla" inicial.	
Entregar	
Asignaturas	Estudiantes

Figura 22: Mock up 14

LOGO		Bienvenido, José Manuel (Usuario) Cerrar sesión	
Imagen de la asignatura			
Sistema operativo			
Prueba de comandos		Fecha de vencimiento: 7/2/2024 20:11	
Descripción			
		Volver	Publicar comentario
Asignaturas		Estudiantes	

Figura 23: Mock up 15

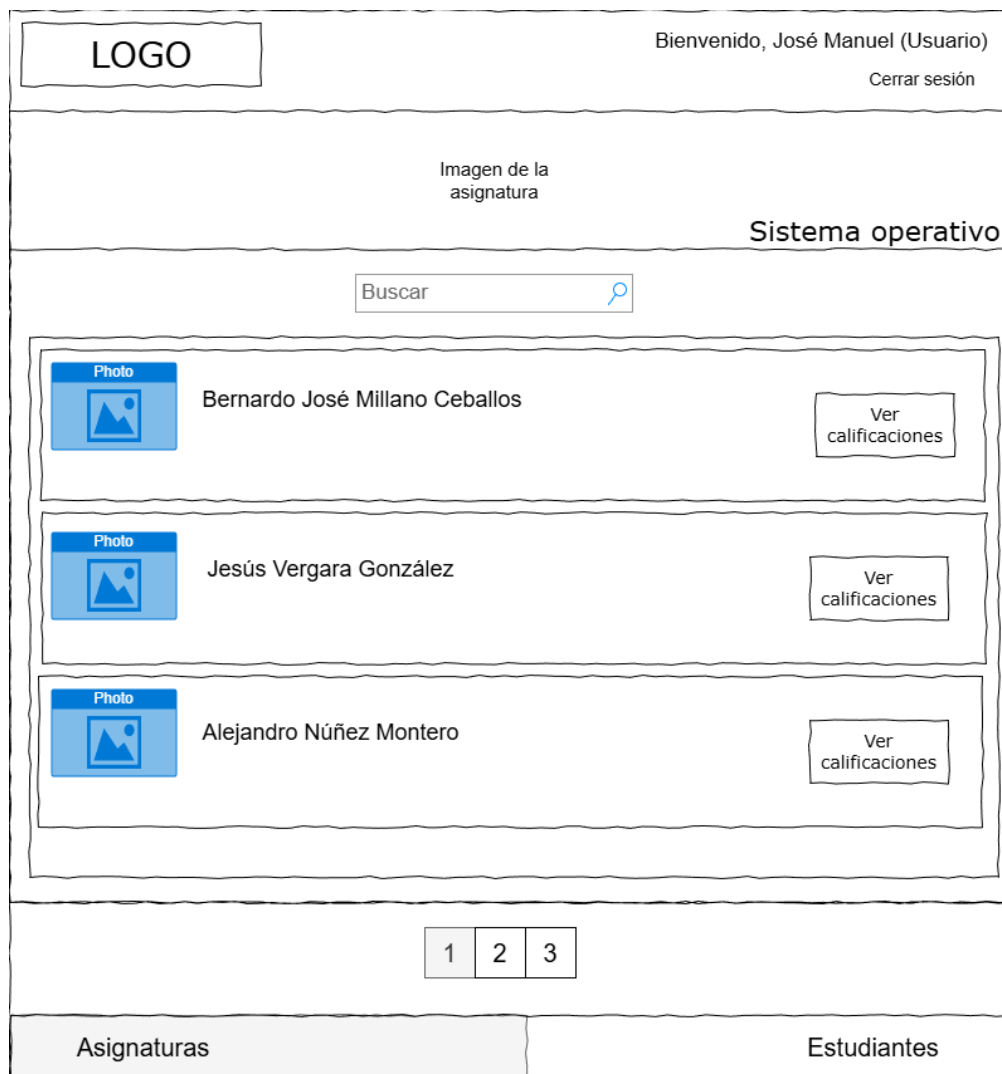


Figura 24: Mock up 16

5.3 Descripción de la interfaz de usuario del sistema

- Mock up 1: Inicio de sesión, donde debes poner tu nombre de usuario y contraseña para acceder. También puedes ir a la página de registro
- Mock up 2: Página de registro, donde debes completar formulario poniendo nombre, correo, teléfono y contraseña.
- Mock up 3: En esta vista, los estudiantes podran ver en que cursos se encuentran, pudiendo seleccionar uno para obtener más detalles de los mismos.
- Mock up 4: En esta vista se ve como vería los entregables un estudiante dentro de la asignatura. Además de poder cambiar a otra vista para ver las calificaciones del curso.
- Mock up 5: En esta vista se ve como vería los entregables un estudiante dentro de la asignatura habiendo seleccionado que se quieren ver los entregables que no entrego a tiempo. Además de poder cambiar a otra vista para ver las calificaciones del curso.
- Mock up 6: En esta vista el estudiante puede ver la calificación conseguida de cada actividad, pudiendo entrar a cada una en caso de que se quiera ver el feedback.
- Mock up 7: En esta vista se enseña como vería la aplicación el profesor nada más loguearse, pudiendo meterse en cada curso.
- Mock up 8: En esta vista se ve un curso desde la vista de un profesor, esto permite ver que número de alumnos han entregado ya la asignatura, permite ir a editar las actividades o ver las entregas de los alumnos. Además de tener la posibilidad de filtrar los entregables por grupo
- Mock up 9: En esta vista se puede crear una actividad, eligiendo si se quiere que sea oculto o visible, poner el título del mismo, establecer una descripción, poner como se va a ver la puntuación de la entrega, la nota máxima que se puede sacar, establecer la fecha limite que se puede entregar y por último seleccionar los grupos a los que se quiere poner la actividad.
- Mock up 10: En esta vista se puede modificar una actividad, eligiendo si se quiere que sea oculto o visible, poner el título del mismo, establecer una descripción, poner como se va a ver la puntuación de la entrega, la nota máxima que se puede sacar, establecer la fecha limite que se puede entregar y por último seleccionar los grupos a los que se quiere poner la actividad. Además de poder eliminarse, saliendo una alerta de confirmación.
- Mock up 11: En esta vista, el profesor puede seleccionar un estudiante para calificarlo además de ponerle feedback en caso de que lo vea necesario de un entregable seleccionado con anterioridad.
- Mock up 12: En esta vista un profesor puede tanto asignar una calificación por entregable al estudiante como ponerle feedback.

- Mock up 13: En esta vista se ve como se vería la entrega de un estudiante nada más abrirla.
- Mock up 14: En esta vista se ve como se vería la entrega de un estudiante poniendo el elemento a entregar viendo una previsualización del mismo, además de poder eliminar el archivo entregado en caso de fallo.
- Mock up 15: En esta vista se puede ver como el profesor puede poner feedback al estudiante.
- Mock up 16: En esta vista puede ver el profesor las clasificaciones sacadas de un alumno en la asignatura.

6 Interfaz de servicios del sistema

6.1 Diagramas de la interfaz de servicios del sistema

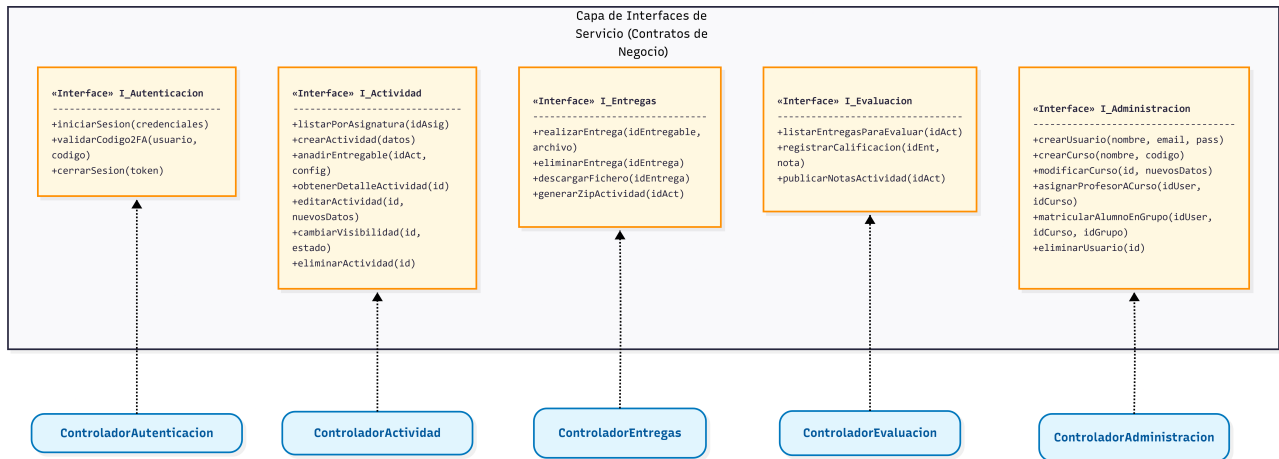


Figura 25: Diagrama de Interfaz

6.2 Descripción de la interfaz de servicios del sistema

OP	SYSOP-001	iniciarSesion
<pre> /** Esta operación inicia el proceso de autenticación validando las credenciales contra el directorio institucional (UVUS). Si son correctas, solicita el segundo factor. */ sysop iniciarSesion : String(usuario: string // identificador o correo institucional contraseña: string // credenciales de acceso) { precondition: ◦ No haber iniciado sesión previamente postcondition: ◦ Si las credenciales son válidas, el sistema queda a la espera del código 2FA ◦ Si no, devuelve error de autenticación } </pre>		
OP	SYSOP-002	validarCodigo2FA
<pre> /** Esta operación completa el inicio de sesión verificando el código temporal proporcionado. Genera el token de sesión final con los roles asociados. */ sysop validarCodigo2FA : Token(usuario: string // identificador del usuario codigo: Integer // código numérico temporal de 6 dígitos) { precondition: ◦ Haber superado la validación de credenciales (SYSOP-001) ◦ El código no debe haber expirado postcondition: ◦ Generación de token JWT de sesión ◦ Redirección al panel principal según el rol (Alumno/Profesor) } </pre>		

op SYSOP-003	cerrarSesion
<pre> /** Invalida el token de sesión actual del usuario, impidiendo futuras peticiones hasta que se vuelva a autenticar. Es fundamental por seguridad. */ sysop cerrarSesion : void(token: string // token JWT actual) { precondition: ◦ Tener una sesión activa postcondition: ◦ Elimina la sesión del almacenamiento temporal (lista negra de tokens o expiración) ◦ Redirige a la pantalla de login } </pre>	

op SYSOP-004	listarActividades
<pre> /** Devuelve el listado de actividades de una asignatura. Si quien consulta es alumno, solo devuelve las visibles. Si es profesor, devuelve todas. */ sysop listarActividades : List<Actividad>(idAsignatura: integer // Identificador de la asignatura) { precondition: ◦ Estar autenticado y matriculado/asignado a la asignatura postcondition: ◦ Lista filtrada según el rol del usuario solicitante } </pre>	

op SYSOP-005	crearActividad
<pre> /** Esta operación permite a un profesor definir una nueva actividad en una asignatura. Adicionalmente, permite adjuntar una lista de archivos de referencia (guías, plantillas) para que los alumnos los descarguen. */ sysop crearActividad : Actividad(idAsignatura: integer // identificador del curso asociado descripcion: string // nombre de la actividad descripcion: string // instrucciones para el alumno fechaInicio: DateTime // fecha y hora de apertura de actividad fechaLimite: DateTime // fecha y hora de cierre de la actividad Visibilidad: boolean // true = Publicada inmediatamente, false = Oculto para los alumnos materialAdjunto: List // lista de documentos de soporte) { precondition: ◦ Estar autenticado como Profesor ◦ La fecha de inicio debe ser anterior a la fecha límite postcondition: ◦ Crea la Actividad en estado "Borrador" (Oculto). ◦ Redirige a la pantalla de detalles de la actividad para poder añadirle entregables } </pre>	

op SYSOP-006	anadirEntregable
<pre> /** Define un requisito de entrega dentro de una actividad. Aquí se especifican las restricciones técnicas (formato y peso) para el alumno. Ejemplo: Un entregable "Memoria PDF" y otro "Código ZIP". */ sysop anadirEntregable : Entregable(idActividad: integer // Actividad a la que pertenece nombre: string // Ej: "Memoria del Proyecto" formatosPermitidos: List // Ej: [".pdf", ".docx"] pesoMaximo: long // Ej: 20MB) { precondition: ◦ Usuario PROFESOR propietario de la actividad. - La actividad debe existir. postcondition: ◦ Crea un "hueco" de entrega (Entregable) asociado a la actividad. - Configura las reglas de validación que usará el SPA. } </pre>	

op	SYSOP-007	obtenerPorId
<pre> /** Recupera toda la información de una actividad específica para mostrarla en pantalla. */ sysop obtenerPorId : ActividadDTO(idActividad: string // Identificador único de la actividad a consultar) { precondition: ◦ El usuario debe estar autenticado. ◦ LÓGICA DE ACCESO: ▪ a) Si es PROFESOR: Debe tener permisos sobre la asignatura (independientemente del estado). ▪ b) Si es ALUMNO: Debe estar matriculado Y la actividad debe tener 'visible = true'. postcondition: ◦ Retorna el objeto con: Datos generales, Enlaces a material adjunto y Estado de la entrega del alumno (si aplica). } </pre>		

op	SYSOP-008	Editar Actividad
<pre> /** Permite modificar los datos de una actividad existente. Es necesario validar que la nueva fecha de inicio no sea posterior a la de fin y que, si la actividad ya tiene entregas, no se cambien condiciones críticas. */ sysop Editar Actividad : void(idActividad: integer // identificador de la actividad a modificar titulo: string // descripción descripcion: string // descripción fechaInicio: DateTime // descripción fechaLimite: DateTime // descripción visible: boolean // descripción materialAdjunto: List // descripción) { precondition: ◦ Estar autenticado como Profesor ◦ Ser el propietario de la actividad ◦ La nueva fechaInicio < nueva fechaLimite postcondition: ◦ Actualiza los datos en la base de datos ◦ Si se añaden nuevos archivos al material adjunto, los sube y vincula } </pre>		

op	SYSOP-009	cambiarVisibilidad
<pre> /** Esta operación cambia el estado de una actividad para hacerla visible u oculta para los alumnos matriculados. */ sysop cambiarVisibilidad : void(idActividad: integer // identificador de la actividad visible: boolean // true para publicar, false para ocultar) { precondition: ◦ Estar autenticado como Profesor ◦ Ser el propietario de la actividad postcondition: ◦ Actualiza el estado de visibilidad en la base de datos } </pre>		

op SYSOP-010	realizarEntrega
<pre> /** El alumno sube un archivo para cumplir con un ENTREGABLE específico. */ sysop realizarEntrega : void(idAsignatura: integer // identificador del curso archivo: File // flujo de bytes del documento metadatos: MetaDTO // tamaño, nombre original y extensión) { precondition: ◦ Usuario ALUMNO matriculado. ◦ La Actividad contenedora debe estar VISIBLE y en PLAZO. ◦ VALIDACIÓN TÉCNICA: El archivo debe coincidir con los 'formatosPermitidos' y 'pesoMaximo' definidos en el SYSOP-006 para este entregable. postcondition: ◦ Archivo renombrado y guardado físicamente. ◦ Se registra la entrega vinculándola al alumno y al entregable concreto. } </pre>	

op SYSOP-011	eliminarActividad
<pre> /** Elimina una actividad del sistema. RESTRICCIÓN DE SEGURIDAD: ◦ Si existen entregas de alumnos (ficheros) pero NO están calificadas, el sistema permite el borrado en cascada (elimina actividad y ficheros). ◦ Si existen calificaciones o feedback registrado, el sistema BLOQUEA la operación para proteger la integridad de las notas. */ sysop eliminarActividad : void(idActividad: integer // identificador de la actividad a borrar) { precondition: ◦ Estar autenticado como Profesor ◦ Ser propietario de la actividad ◦ La actividad debe existir ◦ NO tener calificaciones ni feedback registrados en el sistema (Integridad Académica) postcondition: ◦ Elimina todos los registros de entregas (archivos) de los alumnos. ◦ Ordena al SPA (Sistema de Archivos) borrar físicamente la carpeta de la actividad. ◦ Elimina el registro de la actividad de la base de datos. } </pre>	

op SYSOP-012	eliminarEntrega
<pre> /** Permite al alumno eliminar su entrega si se ha equivocado. */ sysop eliminarEntrega : void(idEntrega: integer // ID de la entrega a cancelar) { precondition: ◦ El usuario debe ser el autor de la entrega. ◦ La actividad asociada debe seguir abierta (en plazo). ◦ La entrega NO debe haber sido calificada por el profesor. postcondition: ◦ Se marca el registro como eliminado o se borra físicamente. ◦ El estado de la entrega del alumno vuelve a "Pendiente". } </pre>	

op	SYSOP-013	descargarFichero
<pre> /** Permite la descarga de un fichero específico (ya sea una entrega de un alumno o un material adjunto del profesor). El sistema verifica que el usuario tenga permisos para ver ese archivo concreto. */ sysop descargarFichero : File(idFichero: integer // identificador único del archivo en base de datos) { precondition: ◦ Estar autenticado ◦ Si es alumno: ser el autor de la entrega o que sea material público ◦ Si es profesor: tener acceso al curso postcondition: ◦ Devuelve el flujo de bytes del archivo solicitado } </pre>		

op	SYSOP-014	generarZipActividad
<pre> /** Genera un archivo comprimido (ZIP) que contiene todas las entregas realizadas para una actividad específica, facilitando la corrección offline. */ sysop generarZipActividad : File(idActividad: integer // identificador de la actividad)) { precondition: ◦ Estar autenticado como Profesor ◦ Existir entregas asociadas a la actividad postcondition: ◦ Devuelve un flujo de bytes correspondiente al archivo ZIP generado } </pre>		

op	SYSOP-015	listarEntregasParaEvaluar
<pre> /** Proporciona al profesor el listado de alumnos y el estado de sus entregas. */ sysop listarEntregasParaEvaluar : List<EntregaResumenDTO>(idActividad: integer // Actividad a evaluar) { precondition: ◦ Usuario PROFESOR con acceso a la asignatura. postcondition: ◦ Devuelve una lista con: Datos del alumno, Fecha de entrega (o indicación de "No entregado"), Estado de corrección (Pendiente/Calificado) y Nota actual. } </pre>		

op	SYSOP-016	registrarCalificacion
<pre> /** Permite al profesor asignar una calificación numérica y un comentario de feedback a una entrega específica. */ sysop registrarCalificacion : void(idEntrega: integer // identificador de la entrega del alumno nota: double // calificación numérica (0-10) comentarios: string // descripción) { precondition: ◦ Usuario PROFESOR. ◦ La entrega debe existir en el sistema. ◦ El valor de la nota debe estar dentro del rango permitido por la configuración de la asignatura. postcondition: ◦ Se actualiza el registro de la entrega con la nota y la fecha de corrección. ◦ El estado interno cambia a 'Calificado' (pero no necesariamente visible para el alumno). } </pre>		

op	SYSOP-017	publicarNotasActividad
<pre> /** Hace visibles las notas y comentarios para todos los alumnos de una actividad. */ sysop publicarNotasActividad : void(idActividad: integer // identificador de la actividad) { precondition: ◦ Usuario PROFESOR. ◦ Deben existir calificaciones registradas en estado "Borrador/Oculto". postcondition: ◦ Todas las calificaciones asociadas a la actividad cambian de estado a 'PUBLICADO'. ◦ Los alumnos pueden ver sus notas y feedback a través de SYSOP-007. ◦ Se dispara un evento de notificación (email/push) a los alumnos afectados. } </pre>		

op	SYSOP-018	crearUsuario
<pre> /** Permite al administrador dar de alta manualmente a un nuevo usuario cuando no se usa el alta automática por OAuth/Google. */ sysop crearUsuario : void(nombre: string // descripción email: string // descripción password: string // descripción) { precondition: ◦ Estar autenticado con rol de ADMINISTRADOR. - El email no debe existir previamente en la base de datos. postcondition: ◦ Se crea el registro de identidad en la base de datos. } </pre>		

op	SYSOP-019	crearCurso
<pre> /** Crea una asignatura o curso en el sistema. Permite la posterior creación de grupos y asignación de personal docente. */ sysop crearCurso(nombre: string // descripción codigo: string // descripción) { precondition: ◦ Estar autenticado como ADMINISTRADOR. ◦ El código del curso debe ser único. postcondition: ◦ Registra la nueva asignatura en la base de datos. } </pre>		

op	SYSOP-020	modificarCurso
<pre> /** Permite al administrador modificar los datos identificativos de un curso existente, como su nombre o su código de referencia. */ sysop modificarCurso : void(idCurso: integer // descripción nuevoNombre: string // descripción nuevoCodigo: string // descripción) { precondition: ◦ Estar autenticado como ADMINISTRADOR. ◦ El curso debe existir. ◦ El nuevo código, si se cambia, no debe estar en uso por otro curso. postcondition: ◦ Actualiza los metadatos del curso en la base de datos. } </pre>		

op	SYSOP-021	asignarProfesorACurso
<pre> /** * Vincula a un usuario con un curso bajo el rol de PROFESOR. Esto le otorga permisos de edición y * evaluación sobre todas las actividades y entregables de dicho curso. Un mismo usuario puede ser asignado * como profesor aquí aunque sea alumno en otros cursos. */ sysop asignarProfesorACurso : void(idUsuario: integer // descripción idCurso: integer // descripción) { precondition: ◦ Estar autenticado como ADMINISTRADOR. ◦ El usuario y el curso deben existir. postcondition: ◦ Crea una relación de docencia en la base de datos para este contexto específico. } </pre>		

op	SYSOP-022	matricularAlumnoEnGrupo
<pre> /** * Vincula a un usuario con un curso bajo el rol de ALUMNO. La matrícula requiere obligatoriamente la * asignación a un grupo específico. Un mismo usuario puede matricularse como alumno aquí aunque sea * profesor en otros cursos. */ sysop matricularAlumnoEnGrupo : void(idUsuario: integer // descripción idCurso: integer // descripción idGrupo: string // descripción) { precondition: ◦ Estar autenticado como ADMINISTRADOR. ◦ El usuario, el curso y el grupo deben existir. postcondition: ◦ Crea una relación de matrícula vinculada al grupo seleccionado en este curso. } </pre>		

op	SYSOP-023	eliminarUsuario
<pre> /** * Elimina al usuario del sistema, lo que provoca la revocación automática de todas sus matrículas y * asignaciones docentes en todos los contextos. */ sysop eliminarUsuario : void(idUsuario: integer // descripción) { precondition: ◦ Estar autenticado como ADMINISTRADOR. postcondition: ◦ Se eliminan los registros de identidad y todas sus vinculaciones como alumno o profesor. } </pre>		

6.3 Servicios consumidos por el sistema

El sistema requiere la integración con servicios externos para garantizar la autenticación segura, la integridad de los archivos almacenados y la comunicación con los usuarios. A continuación se detallan las APIs y servicios consumidos:

6.3.1 Proveedor de Identidad y Autenticación (Google OAuth 2.0 / OpenID Connect)

Descripción:

El sistema delega la gestión de credenciales y el control de acceso en la plataforma de identidad de Google. Se utiliza el protocolo estándar OAuth 2.0 para la autorización y OpenID Connect para la autenticación. Esto permite que los usuarios inicien sesión utilizando su cuenta institucional (ej: @alum.us.es) o personal, delegando en Google la responsabilidad de la seguridad de la contraseña y la verificación en dos pasos (2FA).

Justificación:

Sustituye al directorio activo local (LDAP) y simplifica los SYSOP-001 (Iniciar Sesión) y SYSOP-002 (Validar 2FA).

- **Seguridad:** Permite que nuestra aplicación no tenga que almacenar o gestionar contraseñas sensibles.
- **Fiabilidad:** Garantiza un uptime del 99.9% para el login.
- **Datos:** Provee de forma segura el email y nombre real del usuario para crear su perfil automáticamente.

Documentación Técnica:

<https://developers.google.com/identity/protocols/oauth2/openid-connect>

6.3.2 Servicio de Análisis de Malware (VirusTotal API)

Descripción:

Servicio API REST que permite el escaneo automático de archivos binarios utilizando más de 70 motores antivirus simultáneos.

Justificación:

Es un servicio crítico consumido durante la operación SYSOP-011 (procesarSubida). Dado que el sistema permite la subida de archivos comprimidos (.zip) y documentos que pueden contener macros, es imprescindible analizar cada fichero en la nube antes de guardarlo en el disco duro del servidor, protegiendo así a los profesores que descargarán esos archivos para corregirlos.

Documentación Técnica:

<https://developers.virustotal.com/reference/overview>

6.3.3 Servicio de Notificaciones (SMTP / SendGrid)

Descripción:

Plataforma de entrega de correo electrónico transaccional que permite el envío masivo y fiable de notificaciones.

Justificación:

El sistema consume este servicio para completar el ciclo de comunicación en los SYSOP-016 (publicarNotasActividad) y avisos de mantenimiento. Garantiza que los correos de notas o cambios de fecha lleguen a la bandeja de entrada del alumno y no a la carpeta de SPAM, algo frecuente si se usa un servidor de correo local básico.

6.3.4 Servicio de Sincronización Horaria (NTP)

Descripción:

Consumo de servicios de tiempo de red (Network Time Protocol) fiables (ej. pool.ntp.org).

Justificación:

Fundamental para la integridad de los SYSOP-005 (crearActividad) y SYSOP-011 (procesarSubida). En un sistema académico donde un retraso de 1 segundo puede marcar una entrega como "Fuera de Plazo", el sistema no puede depender del reloj interno del servidor, que puede desfasarse. Se consume un servicio NTP para garantizar sellados de tiempo (timestamps) irrefutables y precisos.

7 Información sobre trazabilidad

MATR-005 A- A+	RQF-001	RQF-002	RQF-003	RQF-004	RQF-005	RQF-006	RQF-007	RQF-008	RQF-009	RQF-010	RQF-011	RQF-012	RQF-013	RQF-014	RQF-015	RQF-016	RQF-017	RQF-018	RQF-019	RQF-020	RQF-021	RQF-022	RQF-023	RQF-024	RQF-025
SYSOP-001	-	↑	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
SYSOP-002	-	↑	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
SYSOP-003	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
SYSOP-004	-	-	-	-	-	-	-	-	-	↑	-	-	-	-	-	-	-	-	-	-	-	-	↑	-	-
SYSOP-005	-	-	-	-	↑	-	↑	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
SYSOP-006	-	-	-	-	-	-	-	-	-	-	↑	↑	-	-	-	-	-	-	-	-	-	-	-	↑	-
SYSOP-007	-	-	-	-	-	-	-	-	-	↑	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
SYSOP-008	-	-	-	-	-	-	↑	↑	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
SYSOP-009	-	-	-	-	-	↑	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	↑	-
SYSOP-010	-	-	-	-	-	-	-	-	-	-	-	-	-	↑	↑	↑	↑	-	-	-	-	-	-	-	-
SYSOP-011	-	-	-	-	-	-	-	-	↑	-	-	-	↑	-	-	-	-	-	-	-	-	-	-	-	-
SYSOP-012	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	↑	↑	-	-	-	-	-	-	-	-
SYSOP-013	-	-	-	-	-	-	↑	-	-	-	-	-	-	-	-	-	-	-	-	↑	-	-	-	-	-
SYSOP-014	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	↑	-	-	-	-	-
SYSOP-015	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	↑	-	-	-	-	-
SYSOP-016	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	↑	↑	-	-	-	-	-	-
SYSOP-017	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	↑	-	-	-	-	-	-	-
SYSOP-018	↑	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
SYSOP-019	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	↑	-	-	-	-
SYSOP-020	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	↑	-	-	-
SYSOP-021	-	-	↑	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
SYSOP-022	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
SYSOP-023	-	-	-	↑	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

Matriz de trazabilidad 1: Matriz de trazabilidad de requisitos funcionales con operaciones del sistema.

A Glosario de acrónimos y abreviaturas